

## Week 6 Skill

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir command_history
divya@Divyasree:~$ cd command_history
divya@Divyasree:~/command_history$ pwd
/home/divya/command_history
divya@Divyasree:~/command_history$ nano history_manager.c
divya@Divyasree:~/command_history$ gcc history_manager.c -o history_manager
divya@Divyasree:~/command_history$ ls
history_manager  history_manager.c
divya@Divyasree:~/command_history$ ./history_manager
===== Command History Manager =====
Maximum history capacity: 5 commands

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> add ls
Command stored successfully.

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> add pwd
Command stored successfully.
```

```
Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> add date
Command stored successfully.

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> history

===== Command History =====
1: ls
2: pwd
3: whoami
4: date
=====

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program
```

```

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> get 10
Invalid history entry.

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> check
History buffer is CONSISTENT.
Number of stored commands: 5
Next storage position: 0

Commands:
add <command> - Store command
history       - Display history
get <number>  - Retrieve entry
check        - Validate consistency
exit         - Exit program

> exit
Exiting History Manager...
divya@Divyasree:~/command_history$ |

```

Code:

```

GNU nano 7.2 history_manag
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_COMMAND 100

char history[MAX_HISTORY][MAX_COMMAND];

int count = 0;
int next_position = 0;

/* Store a command in the history buffer */
void store_command(const char *command)
{
    strncpy(history[next_position], command, MAX_COMMAND - 1);
    history[next_position][MAX_COMMAND - 1] = '\0';

    next_position = (next_position + 1) % MAX_HISTORY;

    if (count < MAX_HISTORY)
        count++;

    printf("Command stored successfully.\n");
}

/* Display all commands */
void display_history()
{

```

```

void display_history()
{
    if (count == 0)
    {
        printf("History is empty.\n");
        return;
    }

    printf("\n==== Command History =====\n");

    int start;

    if (count < MAX_HISTORY)
        start = 0;
    else
        start = next_position;

    for (int i = 0; i < count; i++)
    {
        int index = (start + i) % MAX_HISTORY;

        printf("%d: %s\n", i + 1, history[index]);
    }

    printf("===== \n");
}

/* Retrieve a specific history entry */
void retrieve_entry(int number)

```

```

    printf("Invalid history entry.\n");
    return;
}

int start;

if (count < MAX_HISTORY)
    start = 0;
else
    start = next_position;

int index = (start + number - 1) % MAX_HISTORY;

printf("History entry %d: %s\n", number, history[index]);
}

/* Check buffer consistency */
void validate_consistency()
{
    if (count < 0 || count > MAX_HISTORY)
    {
        printf("History buffer is INCONSISTENT.\n");
        return;
    }

    if (next_position < 0 || next_position >= MAX_HISTORY)
    {
        printf("History buffer position is INCONSISTENT.\n");
        return;
    }
}

```

```

GNU nano 7.2                                     history_manager.c
{
    char input[MAX_COMMAND];

    printf("==== Command History Manager ====\n");
    printf("Maximum history capacity: %d commands\n", MAX_HISTORY);

    while (1)
    {
        printf("\nCommands:\n");
        printf("add <command>    - Store command\n");
        printf("history                  - Display history\n");
        printf("get <number>              - Retrieve entry\n");
        printf("check                    - Validate consistency\n");
        printf("exit                      - Exit program\n");

        printf("\n> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting History Manager...\n");
            break;
        }

        else if (strcmp(input, "history") == 0)

            validate_consistency();
    }

    else if (strncmp(input, "add ", 4) == 0)
    {
        char *command = input + 4;

        if (strlen(command) == 0)
        {
            printf("Cannot store an empty command.\n");
        }
        else
        {
            store_command(command);
        }
    }

    else if (strncmp(input, "get ", 4) == 0)
    {
        int number;

        if (sscanf(input + 4, "%d", &number) == 1)
        {
            retrieve_entry(number);
        }
        else
        {
            printf("Invalid entry number.\n");
        }
    }
}

```

```

        else if (strcmp(input, "get ", 4) == 0)
        {
            int number;

            if (sscanf(input + 4, "%d", &number) == 1)
            {
                retrieve_entry(number);
            }
            else
            {
                printf("Invalid entry number.\n");
            }
        }

        else
        {
            printf("Unknown command.\n");
        }
    }

    return 0;
}

```

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir pipeline_project
divya@Divyasree:~$ cd pipeline_project
divya@Divyasree:~/pipeline_project$ pwd
/home/divya/pipeline_project
divya@Divyasree:~/pipeline_project$ nano pipeline_manager.c
divya@Divyasree:~/pipeline_project$ gcc pipeline_manager.c -o pipeline_manager
divya@Divyasree:~/pipeline_project$ ls
pipeline_manager  pipeline_manager.c
divya@Divyasree:~/pipeline_project$ ./pipeline_manager
===== Pipeline Manager =====

Enter first command: ls -l
Enter second command: grep .c

===== Pipeline Layout =====
[1] ls -l | [2] grep .c
=====
Pipeline layout is VALID.

Executing pipeline...
-rw-r--r-- 1 divya divya 4852 Sep 27 09:07 pipeline_manager.c

Pipeline execution completed.
divya@Divyasree:~/pipeline_project$ ./pipeline_manager
===== Pipeline Manager =====

Enter first command: ls
Enter second command: grep xyz

===== Pipeline Layout =====
[1] ls | [2] grep xyz
=====

```

```
[1] ls | [2] grep xyz
=====
Pipeline layout is VALID.

Executing pipeline...

Pipeline execution completed.
divya@Divyasree:~/pipeline_project$ |
```

Code:

```
GNU nano 7.2 pipeline_manager.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 5
#define MAX_LENGTH 100

typedef struct
{
    char command[MAX_LENGTH];
} PipelineCommand;

typedef struct
{
    PipelineCommand commands[MAX_COMMANDS];
    int count;
} Pipeline;

/* Initialize pipeline */
void initialize_pipeline(Pipeline *pipeline)
{
    pipeline->count = 0;
}

/* Store a command */
int add_command(Pipeline *pipeline, const char *command)
{

```

```

    if (pipeline->count >= MAX_COMMANDS)
    {
        printf("Error: Pipeline capacity exceeded.\n");
        return 0;
    }

    strncpy(
        pipeline->commands[pipeline->count].command,
        command,
        MAX_LENGTH - 1
    );

    pipeline->commands[pipeline->count].command[MAX_LENGTH - 1] = '\0';

    pipeline->count++;

    return 1;
}

/* Display pipeline layout */
void display_pipeline(Pipeline *pipeline)
{
    printf("\n==== Pipeline Layout =====\n");

    for (int i = 0; i < pipeline->count; i++)
    {
        printf("[%d] %s", i + 1, pipeline->commands[i].command);

        printf(" | ");
    }

    printf("\n=====");
}

/* Validate pipeline */
int validate_pipeline(Pipeline *pipeline)
{
    if (pipeline->count < 2)
    {
        printf("Invalid pipeline: At least 2 commands are required.\n");
        return 0;
    }

    if (pipeline->count > MAX_COMMANDS)
    {
        printf("Invalid pipeline: Too many commands.\n");
        return 0;
    }

    for (int i = 0; i < pipeline->count; i++)
    {
        if (strlen(pipeline->commands[i].command) == 0)
        {
            printf("Invalid pipeline: Empty command found.\n");
            return 0;
        }
    }
}

```

```

printf("Pipeline layout is VALID.\n");
return 1;
}

/* Execute a two-command pipeline */
void execute_pipeline(Pipeline *pipeline)
{
    if (pipeline->count != 2)
    {
        printf("Execution currently supports exactly 2 commands.\n");
        return;
    }

    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return;
    }

    pid_t pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return;
    }

    perror("execvp");
    exit(1);
}

pid_t pid2 = fork();

if (pid2 < 0)
{
    perror("fork");
    return;
}

if (pid2 == 0)
{
    /* Second child */

    close(pipefd[1]);

    /* Connect stdin to pipe */
    dup2(pipefd[0], STDIN_FILENO);

    close(pipefd[0]);

    char command2[MAX_LENGTH];
    strncpy(
        command2,
        pipeline->commands[1].command,
        MAX_LENGTH - 1
    );
};

```

```
GNU nano 7.2 pipeline_manager.c
char *token = strtok(command2, " ");

while (token != NULL && count < 19)
{
    args[count++] = token;
    token = strtok(NULL, " ");
}

args[count] = NULL;

execvp(args[0], args);

perror("execvp");
exit(1);
}

/* Parent process */
close(pipefd[0]);
close(pipefd[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("\nPipeline execution completed.\n");
}

int main()
{
    Pipeline pipeline;

    command[strcspn(command, "\n")] = '\0';

    if (!add_command(&pipeline, command))
        return 1;

    printf("Enter second command: ");

    if (fgets(command, sizeof(command), stdin) == NULL)
        return 1;

    command[strcspn(command, "\n")] = '\0';

    if (!add_command(&pipeline, command))
        return 1;

    /* Display stored commands */
    display_pipeline(&pipeline);

    /* Validate pipeline */
    if (!validate_pipeline(&pipeline))
    {
        printf("Pipeline execution cancelled.\n");
        return 1;
    }

    printf("\nExecuting pipeline...\n");

    execute_pipeline(&pipeline);
}

printf("\nExecuting pipeline...\n");

execute_pipeline(&pipeline);

return 0;
}
```

TO Create Pipes, Redirect Streams, Execute Connected Commands,  
Close Descriptors, Synchronize Processes, Test Data Flow

```

divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir pipe_dataflow
divya@Divyasree:~$ cd pipe_dataflow
divya@Divyasree:~/pipe_dataflow$ pwd
/home/divya/pipe_dataflow
divya@Divyasree:~/pipe_dataflow$ nano pipe_dataflow.c
divya@Divyasree:~/pipe_dataflow$ gcc pipe_dataflow.c -o pipe_dataflow
divya@Divyasree:~/pipe_dataflow$ ls
pipe_dataflow  pipe_dataflow.c
divya@Divyasree:~/pipe_dataflow$ ./pipe_dataflow
===== Pipe Data Flow Manager =====
Pipe created successfully.
Read descriptor : 3
Write descriptor : 4

[Child 1] Executing echo command...

[Parent] Closing unused pipe descriptors...
[Parent] Waiting for child processes...
4
[Parent] Both child processes completed.
===== Data Flow Test Completed =====
divya@Divyasree:~/pipe_dataflow$ pipe(pipefd);
-bash: syntax error near unexpected token `pipefd'
divya@Divyasree:~/pipe_dataflow$ wc -w
|

```

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    int pipefd[2];

    printf("===== Pipe Data Flow Manager =====\n");

    /* Create pipe */
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    printf("Pipe created successfully.\n");
    printf("Read descriptor : %d\n", pipefd[0]);
    printf("Write descriptor : %d\n", pipefd[1]);

    /* Create first child */
    pid_t pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
    }
}

```

```
GNU nano 7.2 pipe_dataflow.c
}

if (pid1 == 0)
{
    /* First child */

    printf("\n[Child 1] Executing echo command...\n");

    /*
     * Redirect standard output
     * to the write end of pipe
     */
    if (dup2(pipefd[1], STDOUT_FILENO) == -1)
    {
        perror("dup2");
        exit(1);
    }

    /* Close unused descriptors */
    close(pipefd[0]);
    close(pipefd[1]);

    execlp("echo", "echo", "Hello Linux Pipe Data", NULL);

    perror("execlp");
    exit(1);
}

/* Create second child */

GNU nano 7.2 pipe_dataflow.c
if (pid2 < 0)
{
    perror("fork");
    return 1;
}

if (pid2 == 0)
{
    /* Second child */

    /*
     * Redirect standard input
     * to the read end of pipe
     */
    if (dup2(pipefd[0], STDIN_FILENO) == -1)
    {
        perror("dup2");
        exit(1);
    }

    /* Close unused descriptors */
    close(pipefd[1]);
    close(pipefd[0]);

    /*
     * wc -w counts the number of words
     */
    execlp("wc", "wc", "-w", NULL);

    perror("execlp");
    exit(1);
}

/* Parent process */

printf("\n[Parent] Closing unused pipe descriptors...\n");

close(pipefd[0]);
close(pipefd[1]);

printf("[Parent] Waiting for child processes...\n");

/* Synchronize processes */
waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);

printf("[Parent] Both child processes completed.\n");
printf("==== Data Flow Test Completed =====\n");

return 0;
}
```

## To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines

```
divya@Divyasree:~$ cd
divya@Divyasree:~$ mkdir multi_pipe_project
divya@Divyasree:~$ cd multi_pipe_project
divya@Divyasree:~/multi_pipe_project$ pwd
/home/divya/multi_pipe_project
divya@Divyasree:~/multi_pipe_project$ nano multi_pipe.c
divya@Divyasree:~/multi_pipe_project$ gcc multi_pipe.c -o multi_pipe
divya@Divyasree:~/multi_pipe_project$ ls
multi_pipe  multi_pipe.c
divya@Divyasree:~/multi_pipe_project$ ./multi_pipe
===== Multiple Pipe Manager =====
Number of commands: 4
Number of pipes: 3
Pipe 1 created: read=3 write=4
Pipe 2 created: read=5 write=6
Pipe 3 created: read=7 write=8
[Child 1] Starting command: ls
[Child 2] Starting command: grep

[Parent] Closing pipe descriptors...
[Child 3] Starting command: wc
[Parent] Waiting for all child processes...
[Child 4] Starting command: cat
1

===== Long Pipeline Completed =====
divya@Divyasree:~/multi_pipe_project$ ls -l
total 24
-rwxr-xr-x 1 divya divya 17088 Sep 27 09:41 multi_pipe
-rw-r--r-- 1 divya divya 3255 Sep 27 09:40 multi_pipe.c
divya@Divyasree:~/multi_pipe_project$ grep .c
^C
divya@Divyasree:~/multi_pipe_project$ wc -l
^C

===== Long Pipeline Completed =====
divya@Divyasree:~/multi_pipe_project$ ls -l
total 24
-rwxr-xr-x 1 divya divya 17088 Sep 27 09:41 multi_pipe
-rw-r--r-- 1 divya divya 3255 Sep 27 09:40 multi_pipe.c
divya@Divyasree:~/multi_pipe_project$ grep .c
^C
divya@Divyasree:~/multi_pipe_project$ wc -l
^C
divya@Divyasree:~/multi_pipe_project$ cat
^C
divya@Divyasree:~/multi_pipe_project$ ./multi_pipe
===== Multiple Pipe Manager =====
Number of commands: 4
Number of pipes: 3
Pipe 1 created: read=3 write=4
Pipe 2 created: read=5 write=6
Pipe 3 created: read=7 write=8
[Child 1] Starting command: ls
[Child 2] Starting command: grep

[Parent] Closing pipe descriptors...
[Child 3] Starting command: wc
[Parent] Waiting for all child processes...
[Child 4] Starting command: cat
1

===== Long Pipeline Completed =====
divya@Divyasree:~/multi_pipe_project$ |
```

```

GNU nano 7.2 multi_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define NUM_COMMANDS 4
#define MAX_ARGS 20
#define MAX_LENGTH 100

/*
    Pipeline:

    ls -l
    |
    v
    grep .c
    |
    v
    wc -l
    |
    v
    cat
*/

char *commands[NUM_COMMANDS][MAX_ARGS] =
{
    {"ls", "-l", NULL},
    {"grep", ".c", NULL},
}

GNU nano 7.2 multi_pipe.c
{
    /*
        Number of pipes required for N commands:

        N - 1 pipes

        For 4 commands:
        4 - 1 = 3 pipes
    */

    int pipes[NUM_COMMANDS - 1][2];

    pid_t pids[NUM_COMMANDS];

    printf("==== Multiple Pipe Manager =====\n");
    printf("Number of commands: %d\n", NUM_COMMANDS);
    printf("Number of pipes: %d\n", NUM_COMMANDS - 1);

    /*
        Create multiple pipes
    */

    for (int i = 0; i < NUM_COMMANDS - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            exit(1);
        }
    }
}

```

```
    printf("Pipe %d created: read=%d write=%d\n",
           i + 1,
           pipes[i][0],
           pipes[i][1]);
}

/*
 * Create one process for each command
 */

for (int i = 0; i < NUM_COMMANDS; i++)
{
    pids[i] = fork();

    if (pids[i] < 0)
    {
        perror("fork");
        exit(1);
    }

    /*
     * Child process
     */

    if (pids[i] == 0)
    {
        printf("[Child %d] Starting command: %s\n",
               i + 1,
               commands[i][0]);
```

```
        if (i > 0)
        {
            if (dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
            {
                perror("dup2 stdin");
                exit(1);
            }
        }

        /*
         * If this is not the last command,
         * connect stdout to next pipe.
         */

        if (i < NUM_COMMANDS - 1)
        {
            if (dup2(pipes[i][1], STDOUT_FILENO) == -1)
            {
                perror("dup2 stdout");
                exit(1);
            }
        }

        /*
         * Close ALL pipe descriptors in child
         * after dup2().
         */

        for (int j = 0; j < NUM_COMMANDS - 1; j++)
```

```

    }

    /*
     * Parent process closes all pipe descriptors.
     */

    printf("\n[Parent] Closing pipe descriptors...\n");

    for (int i = 0; i < NUM_COMMANDS - 1; i++)
    {
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    /*
     * Wait for all child processes.
     */

    printf("[Parent] Waiting for all child processes...\n");

    for (int i = 0; i < NUM_COMMANDS; i++)
    {
        waitpid(pids[i], NULL, 0);
    }

    printf("\n==== Long Pipeline Completed ==== \n");

    return 0;
}

```